

Terraform, as the City's Architect

BUILT BEFORE KUBERNETES EVER MOVES IN

THE BIG PICTURE

Kubernetes runs the city — deciding who lives where, keeping apartments filled. But before any of that can happen, the city has to actually **exist**. **Terraform is the architect and construction crew** — it builds the land, the buildings, the roads, before anyone moves in. Terraform builds the city; Kubernetes runs it.

PROVIDER —the construction company you hire

Terraform doesn't build anything itself — it hires a construction company to do it. AWS, Azure, and Google Cloud are all different companies, each with their own way of building. You tell Terraform which one to hire before anything gets built.

Example: "Hire the AWS company for this city." Everything built afterward goes through AWS specifically, using AWS's own methods and pricing.

RESOURCE —one actual thing being built

A resource is one specific item in the blueprint — one server, one network, one piece of storage. Even identical items are counted separately — 3 servers is 3 resources, not 1.

Example: A blueprint asking for 3 servers and 1 network contains 4 resources in total — each one tracked and built individually.

STATE —the architect's own record book

Terraform keeps a record of everything it has already built. Every time it runs again, it checks this record first, compares it to the blueprint, and only builds or changes whatever's actually different.

Example: If the blueprint asks for 5 servers and the record book shows 3 already exist, Terraform builds exactly 2 more — not 5 new ones. If someone deletes a server by hand outside Terraform, the record book no longer matches reality — this is called "drift," and it causes confusing errors later.

PLAN —the preview, before anything is built

Before building anything for real, Terraform can show exactly what it's about to create, change, or destroy — like an architect showing you the final blueprint before construction starts.

Example: You ask for 5 servers instead of 3. Before touching anything, Terraform reports "2 will be created, 0 changed, 0 destroyed" — so nothing destructive ever happens as a surprise.

VARIABLE —a fill-in-the-blank blueprint

Instead of hardcoding details into the blueprint, you leave blanks — "build [this many] servers, of [this] size" — and fill in the actual values separately. One blueprint, many possible outcomes.

Example: The same blueprint builds a small test city ("2 servers, small") or a large production city ("10 servers, large") — just by filling the blanks differently each time.

OUTPUT —information handed back after building

Once something is built, Terraform can hand back useful details automatically — like a new server's address — instead of you having to go dig for it manually.

Example: A new server's address gets printed the moment it's built, so it can be saved to a file for another tool (like Ansible) to immediately pick up and use.

MODULE —a reusable, packaged blueprint

A module packages up a design so it can be reused across projects, instead of rewriting the same thing every time. This is the direct Terraform equivalent of an Ansible role.

Example: If your company always builds servers the same standard way, you build that design once as a module — every future project just reuses it instead of recreating it.

Provider (who builds it) → Resource (what gets built)
→ State (the record of what's real) → Plan (the preview before it happens)
→ Variables (fill-in-the-blank reuse) → Output (info handed back)
→ Module (the whole design, packaged for reuse)

Set up the working directory PROVIDER — CONSTRUCTION COMPANY

```
terraform init
```

Hires the **construction company (provider)** — downloads the plugin Terraform needs to actually talk to AWS/Azure/etc. Run this first, every time, in a new project.

Preview the changes PLAN — THE PREVIEW

```
terraform plan
```

Shows exactly what will be created, changed, or destroyed — **before** anything actually happens. Always run this before apply.

Actually build it RESOURCE — THE THING BEING BUILT

```
terraform apply
```

```
yes
```

— confirms the plan when prompted, and construction begins for real

Builds the **resources** for real, matching whatever the plan showed. This is the only command that actually changes real infrastructure.

Check the record book STATE — RECORD BOOK

```
terraform show
```

Displays everything Terraform believes currently exists, straight from its **record book (state)**.

Fill in the blanks VARIABLE — FILL-IN-THE-BLANK

```
terraform apply -var="instance_count=5"
```

```
-var="..."
```

— fills in one blank in the blueprint at run time, without editing the file itself

Runs the same blueprint, but fills the "instance_count" blank with 5 this time — no need to write a separate blueprint for a different outcome.

Get the handed-back info OUTPUT — INFO HANDED BACK

```
terraform output
```

Prints whatever information was defined as **output** — like a newly built server's address — straight to your screen.

Use a packaged design MODULE — REUSABLE BLUEPRINT

```
module "web_server" {
  source = "../modules/web-server"
}
```

```
source = "..."
```

— where the reusable, packaged blueprint lives

Reuses an existing packaged design (**module**) instead of writing the same setup from scratch — same idea as an Ansible role.

Tear it all down STATE & RESOURCE — UNDO EVERYTHING

```
terraform destroy
```

Safely removes every **resource** Terraform built, checking the **state (record book)** to make sure everything it created gets cleanly torn down, in the right order.

Quick reference

Provider — the construction company (AWS, Azure, etc.)

Resource — one actual thing being built

State — Terraform's own record of what's real

Plan — the preview before anything is built

Variable — a fill-in-the-blank value in the blueprint

Output — information handed back after building

Module — a reusable, packaged blueprint

REMOTE STATE —the record book, kept in a shared safe instead of on one desk

If the record book only lives on one architect's own machine, nobody else can safely use Terraform on the same city — two people could easily build conflicting things without realizing it. Remote state stores that record book somewhere shared, so the whole team is always working from the same true record.

Example: A team stores their state file in a shared cloud storage bucket, so whoever runs Terraform — any team member — always sees the same accurate record of what's real.

WORKSPACE —running the same blueprint against multiple separate cities

Beyond just filling in different variable values, a workspace lets you run the exact same blueprint against entirely separate record books — so test and prod stay fully isolated from each other, not just differently configured.

Example: A "dev" workspace and a "prod" workspace both use the same blueprint file, but each has its own separate state, so changes in one never risk touching the other.

REGISTRY —borrowing someone else's already-designed blueprint

Instead of designing a module from scratch, the Terraform Registry is a public library of ready-made modules other people have already built and shared — for common things like a standard server setup or network layout.

Example: Instead of writing a VPC module by hand, a team pulls a well-tested one straight from the registry and just fills in their own variable values.

Configure remote state REMOTE STATE — SHARED SAFE

```
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "prod/terraform.tfstate"
  }
}
```

backend "s3" — where the shared record book actually lives
key = "..." — which specific record book, if storing several

Tells Terraform to keep its state in shared storage instead of a local file — essential the moment more than one person touches the same infrastructure.

Switch between workspaces WORKSPACE — SEPARATE CITIES

```
terraform workspace new dev
```

```
terraform workspace select prod
```

Creates a new isolated workspace, or switches to an existing one — each with its own separate state, from the same blueprint file.

Use a module from the registry REGISTRY — BORROWED BLUEPRINT

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.0.0"
}
```

source = "..." — the published module's location in the registry
version = "..." — locks to a specific, tested version of that module

Pulls in a ready-made, community-maintained module instead of designing the same common infrastructure pattern from scratch.

main.tf THE WRITTEN-DOWN BLUEPRINT

```
provider "aws" {
  region = "us-east-2"
}

variable "instance_count" {
  default = 3
}

resource "aws_instance" "web" {
  count          = var.instance_count
  ami            = "ami-0f19d220602031aed"
  instance_type = "t2.micro"

  tags = {
    Name = "web-server"
  }
}

output "server_ips" {
  value = aws_instance.web[*].public_ip
}
```

provider "aws" — the construction company being hired, and where to build
variable "instance_count"— a blank left in the blueprint, default filled with 3
resource "aws_instance" "web"— the actual thing being built, named "web"
count = var.instance_count— build that many copies, using the variable's value
output "server_ips"— hand back each server's address once built

One file containing a provider, a variable, a resource, and an output together — this is genuinely what a small real-world blueprint looks like. Run **terraform init**, then **terraform plan**, then **terraform apply** against this exact file.

HOW TERRAFORM AND KUBERNETES ACTUALLY CONNECT

In a real pipeline, these two tools aren't separate stories — they're two steps of the same one. **Terraform runs first** and builds the actual cluster itself (the city) — including the nodes (buildings) it will contain. Once that cluster exists, **Kubernetes takes over** and starts managing what runs inside it — deployments, pods, services, all the things we covered in the Kubernetes guide.

So when your manager's team uses Terraform, part of what they're likely building with it is the Kubernetes cluster itself — the infrastructure Kubernetes then depends on to exist in the first place.

QUICK SELF-TEST

1. What's the difference between a provider and a resource?
2. Why does Terraform keep a state file at all?
3. What could go wrong if someone changes infrastructure manually, outside Terraform?
4. Why run "plan" before "apply," instead of just applying directly?
5. If you wanted the same blueprint to build both a small and a large environment, what would you use?
6. What's the Terraform equivalent of an Ansible role?

IF THE MANAGER ASKS...

"Why does Terraform need a state file?"

So it knows what it already built, without re-scanning everything each time. It compares the blueprint to the state, and only changes whatever's actually different.

"Why always run plan before apply?"

Because infrastructure changes can be destructive and hard to undo — plan shows exactly what will be created, changed, or destroyed, so nothing happens as a surprise.

"How would you reuse the same setup for test and prod?"

Using variables — the blueprint itself stays identical, only the values fed into it change, so test and prod are built from a single source of truth instead of two separate files.